

CoIncGraph: CPU–GPU Cooperative Incremental Processing of Out-of-Memory Streaming Graphs

Anonymous Authors

Abstract—Streaming graph processing is essential for applications such as fraud detection and network monitoring, where relationships evolve continuously. When graphs exceed GPU memory, maintaining topology and updating analytical results efficiently requires minimizing redundant computation and CPU–GPU data movement. We present CoIncGraph, a CPU–GPU cooperative system for incremental processing of out-of-memory streaming graphs. First, a heterogeneous source-isolated graph organization exploits topological locality through independently allocated adjacency blocks and a unified change view, limiting data movement and GPU index maintenance to changed sources. Second, CPU–GPU collaborative compact incremental computation improves concurrency by overlapping CPU preparation of affected incoming adjacency with GPU deletion repair, restricting computation and data transfers to affected vertices. Third, workload-specific optimizations complete deletion repair before insertion processing, seed propagation only from vertices improved by inserted edges, and construct compact worklists as improvements propagate, avoiding graph-wide worklist construction and unnecessary adjacency traversal. Extensive experiments show that CoIncGraph outperforms Ingress, SHGraph, and Grapin across CPU-based, GPU in-memory, and CPU–GPU out-of-memory graph processing, respectively.

Keywords—Streaming graph processing, incremental graph computation, CPU–GPU cooperative processing, out-of-core graph processing

I. INTRODUCTION

Streaming graph processing supports applications such as fraud detection and network monitoring, where evolving relationships require timely analytical results. Unlike static graph processing, it must maintain graph topology and update results as edges are inserted and deleted. Recomputing analytics from scratch after each update batch repeats substantial work on unchanged regions. Incremental processing instead reuses existing results and propagates the effects of updates, offering an opportunity to reduce computation. GPUs provide high parallelism and memory bandwidth for this propagation, but large graphs often exceed GPU memory, requiring efficient coordination between host-resident graph storage and GPU computation.

Existing systems address different aspects of this problem. CPU-based systems such as GraphBolt [1] and Ingress [2] memoize intermediate results or dependencies to reuse unaffected computation. GPU-resident systems such as SHGraph [3] support parallel topology updates through dynamic graph representations, but require the graph to fit in device memory. CPU–GPU cooperative systems such as Grapin [4] extend incremental processing to out-of-memory graphs by maintaining topology in host memory and combining GPU hot-subgraph caching with optimized zero-copy access. These

approaches provide important building blocks, yet efficient out-of-memory incremental processing requires more than reducing recomputation or accelerating graph access alone: topology maintenance, dependency repair, and CPU–GPU data movement must preserve update locality throughout the processing pipeline.

First, local topology changes can incur nonlocal maintenance costs. An edge update directly modifies only its source’s outgoing adjacency, but shared graph layouts may move unrelated adjacency lists during insertion or rebalancing. Such movement also invalidates their GPU adjacency descriptors, expanding both data movement and index maintenance beyond the updated sources. Exploiting topological locality therefore requires a layout that isolates adjacency relocation and a maintenance scheme that consistently coordinates outgoing and incoming adjacency views.

Second, coupling deletion repair with insertion processing can make incremental computation unnecessarily broad. Without phase decoupling, the insertion phase must conservatively add all vertices to the active set to search widely for vertices whose results may need repair, incurring substantial computation as well as large CPU–GPU data-transfer overhead. We decouple the two phases so that, during deletion repair, the CPU supplies reliable replacement predecessors and obtains a stable result before insertion edges are processed. Consequently, both deletion repair and subsequent insertion propagation need to update only results related to the modified topology, greatly reducing the work range, computation, and CPU–GPU transfer time.

Third, update locality alone does not eliminate workload-dependent overhead. In large-diameter graphs, successive improvements may propagate over many rounds and repeatedly scan downstream adjacency. For large update batches, repeated preparation, copying, and sorting across maintenance stages can become a substantial CPU cost. These workloads require complementary optimizations that reduce redundant propagation and reuse batch-level maintenance information.

To address these challenges, we present CoIncGraph, a CPU–GPU cooperative system for incremental processing of out-of-memory streaming graphs. CoIncGraph combines three complementary designs. Its *heterogeneous source-isolated graph organization* stores each source’s adjacency in an independently allocated block and coordinates topology changes through a unified change view, confining adjacency relocation and GPU index maintenance to changed sources. Its *CPU–GPU collaborative compact incremental computation* exploits the locality of dynamic graph updates by decoupling insertion

and deletion processing. With CPU assistance, it confines computation to affected vertices and their dependencies, thereby reducing both the computational workload and data transfer overhead of incremental processing. Finally, its *workload-specific optimizations* use ordered propagation to reduce redundant edge scans and shared batch preparation to avoid repeated CPU work.

Our contributions are as follows:

- 1) We design a heterogeneous source-isolated graph organization that preserves topological locality and coordinates consistent maintenance of host adjacency and GPU indices.
- 2) We develop CPU–GPU collaborative compact incremental computation that exploits dynamic-update locality to limit the work set to affected vertices and their dependencies.
- 3) We introduce ordered propagation and shared batch preparation to reduce redundant work in large-diameter graphs and large update batches.
- 4) We evaluate CoIncGraph on six real-world graphs against Ingress, SHGraph, and Grapin, demonstrating its performance benefits across CPU-based, GPU in-memory, and CPU–GPU out-of-memory baselines.

II. BACKGROUND AND MOTIVATION

A. Background

1) *Streaming Graph Processing*: A graph $G = (V, E)$ consists of vertices V and edges E . Algorithms such as breadth-first search (BFS), single-source shortest paths (SSSP), and connected components (CC) can be expressed as iterative worklist processing: active vertices update their neighbors and generate subsequent worklists until convergence [5]. In streaming graph processing, topology changes are commonly grouped into batches of edge insertions and deletions. Applying a batch ΔG_t transforms G_t into G_{t+1} , whose analytical result is $R_{t+1} = \mathcal{A}(G_{t+1})$. Recomputing this result from scratch after each batch may repeat substantial work on unchanged graph regions.

2) *Streaming Graph Representation*: Streaming graph representations balance update efficiency, traversal locality, and memory overhead. Adjacency-list-based layouts organize neighbors separately for each vertex, enabling localized updates but potentially introducing fragmentation and indirection. CSR instead stores adjacency data contiguously using vertex offsets, providing compact storage and efficient traversal, while topology updates may require costly data movement. Dynamic representations address this trade-off through hierarchical storage, reserved space, or buffered updates. For example, Terrace uses a hierarchical graph container [6], Spruce adopts a compact multilevel structure [7], and LSMGraph combines an update-friendly LSM-tree with multilevel CSR storage [8]. These designs provide different balances between update cost and neighborhood access efficiency.

A representative approach incorporates a packed memory array (PMA) into the graph representation [9], [?]. It stores the adjacency of all sources in a single shared array interleaved with gaps, so that the neighbors of each source remain contiguous and an insertion can usually be absorbed by a

nearby gap. The array is divided into segments organized as an implicit tree, and each level enforces lower and upper density thresholds. As illustrated in Fig. 1(c), an insertion shifts the entries between its position and the nearest gap; when a segment exceeds its density threshold, the entries of an enclosing range are redistributed (rebalancing), and when the whole array becomes full, it is doubled in size and all entries are redistributed (expansion).

3) *Incremental Processing*: Incremental processing reuses the results of the previous graph snapshot and propagates update-induced changes through the affected dependencies. KickStarter uses dependency tracking and trimming to repair invalidated results [10], while GraphBolt tracks dependencies across intermediate values to support incremental processing with synchronous semantics [1]. For monotonic algorithms such as BFS and SSSP, dependency-memoization records for each vertex the parent through which its result was obtained. A deletion invalidates the results of all vertices that depend on the deleted edge; each invalidated result must then be restored from the remaining incoming edges of the vertex. An insertion can only improve results, and the improvement is propagated along outgoing edges. Figs. 1(a)–(b) illustrate how incremental computation repairs results via dependencies when the graph topology changes.

4) *Out-of-Memory CPU–GPU Graph Processing*: Out-of-memory systems keep edge data in host memory and place vertex results, together with per-vertex adjacency descriptors that record the location and degree of each neighbor list, on the GPU. Adjacency is transferred either explicitly after the CPU compacts the active subgraph, as in Subway [11], or on demand through zero-copy access, where GPU threads read pinned host memory with fine-grained PCIe requests, as in EMOGI [12]. Grapin extends this model to streaming graphs: it maintains the dynamic topology on the CPU, performs incremental computation on the GPU, and caches the adjacency of frequently accessed vertices in GPU memory as a hot subgraph [4]. CoIncGraph adopts this execution model, including zero-copy access and hotness-based subgraph caching.

B. Motivation

Out-of-memory streaming graph processing typically keeps the complete topology in host memory and places vertex results on the GPU, exploiting its high memory bandwidth and massive parallelism for computation, while adjacency is fetched from host memory over the PCIe bus on demand. Since PCIe bandwidth is far lower than device memory bandwidth, every adjacency transfer and every update to GPU-side graph views incurs substantial cost. Fortunately, streaming updates exhibit *update locality*: in Twitter (TW) and Friendster (FS), a 100K-edge batch touches only 0.18% and 0.15% of the sources, respectively (Upd. in Table I), so both costs should ideally scale with the changed region. However, we observe that existing designs lose this locality in both graph maintenance and incremental computation.

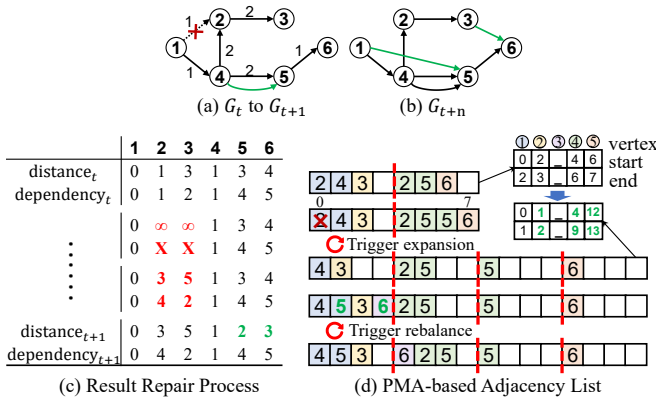


Fig. 1. PMA-based adjacency maintenance and incremental result repair. (a)–(b) An edge update transforms G_t into G_{t+1} and propagates its effect to dependent vertices. (c) In the PMA, local shifts may change the indices of subsequent adjacency entries; insufficient local space triggers expansion, and density violations trigger rebalancing over a larger range. (d) The corresponding vertex results and dependencies are repaired to obtain the state for G_{t+1} .

TABLE I

PER-BATCH STATISTICS OF SSSP (100K-EDGE BATCHES, AVERAGED OVER TEN BATCHES). UPD.: SOURCES TOUCHED BY THE BATCH; DESC.: DESCRIPTOR DATA TRANSFERRED TO THE GPU; INIT. WL: SIZE OF THE INITIAL INSERTION WORKLIST.

Graph	System	Upd.	Desc. (MB)	Init. WL
TW	GrapiN [4]	94,869	841.3	52,579,682
	CoIncGraph	94,869	2.3	1,126
FS	GrapiN [4]	99,188	1,049.7	65,608,366
	CoIncGraph	99,188	2.4	1,684

Graph maintenance amplifies local updates. Shared-array layouts, such as PMA or CSR, keep each source’s adjacency contiguous for coalesced GPU access, but an insertion may shift, rebalance, or expand a range containing sources that receive no update. Since such relocations are decided inside the layout and cannot be derived from the update batch, publishing descriptors only for updated sources would risk leaving relocated sources pointing to stale locations; the descriptors of all $|V|$ vertices are therefore reloaded after every batch. As shown in Table I, this transfers 841 MB and 1,050 MB of descriptors per batch in TW and FS, i.e., 16 bytes for every vertex, although fewer than 0.2% of the sources are touched; publishing only the touched sources, as CoIncGraph does, reduces this volume to about 2 MB. The GPU cache suffers similarly: modifying a cached source invalidates its cached adjacency, and the cache is evicted, compacted, and reloaded after every batch, even when the set of hot vertices barely changes. Consequently, maintenance cost scales with the graph rather than with the changed sources. This calls for a graph organization that confines relocation to updated sources, so that every CPU- and GPU-side graph view can be maintained at the granularity of changed sources.

Incremental computation loses its affected region. Restoring an invalidated vertex v requires one of its remaining incoming edges, e.g., (x, v) , yet the predecessor x is unaf-

fected by the batch and thus inactive, and a GPU engine that traverses outgoing edges cannot locate it. Recovery is therefore merged into insertion processing, which reactivates all vertices and rebuilds worklists by scanning vertex states in every round [4]. As a result, the initial insertion worklist contains all 52.6M and 65.6M vertices in TW and FS, whereas only about 1.1K and 1.7K destinations are directly improved by inserted edges and need to start propagation, which is the initial worklist of CoIncGraph (Table I). The missing incoming adjacency, however, is readily available on the CPU, which maintains the topology in host memory. If the CPU supplies the incoming edges of invalidated vertices, deletion repair can be completed within the affected region before insertions are applied, after which insertion propagation needs to start only from the improved vertices. This motivates decoupling deletion repair from insertion propagation through CPU–GPU cooperation and keeping worklists compact throughout. Sections III-A and III-B present the designs that address these two observations.

III. DESIGN OF COINCGRAPH

Figure 2 presents an overview of CoIncGraph, a CPU–GPU cooperative system for incremental processing of streaming graphs that exceed GPU memory capacity. CoIncGraph integrates two components. First, a heterogeneous graph organization localizes adjacency changes and associated maintenance to modified source vertices. A unified change view coordinates updates to outgoing adjacency, the reverse index, and GPU views, avoiding redundant maintenance across these structures (Section III-A). Second, a two-phase incremental execution pipeline performs deletion repair followed by insertion propagation. Compact worklists identify affected vertices: the CPU prepares incoming edges for GPU-based repair, while the GPU propagates improvements through outgoing edges (Section III-B). Together, these components confine data preparation and traversal to affected states and their dependencies, reducing redundant computation and CPU–GPU communication. In addition, CoIncGraph incorporates workload-specific optimizations to improve performance and broaden its applicability across diverse graph workloads (Section III-B3).

A. Heterogeneous Source-Isolated Graph Organization

Section II-B shows that shared-array layouts transform a local update into the relocation of unrelated sources, resulting in extensive descriptor modifications and per-batch cache reorganization. The key idea of our graph organization is to *make the source vertex of an update edge the unit of storage, change description, and GPU publication*, so that maintenance cost follows the set of changed sources rather than the graph size. It consists of three mechanisms. *Source-isolated adjacency blocks* allocate each source’s adjacency independently, so that an update never relocates other sources. A *unified change view* resolves each batch once into effective edge changes and changed sources that drive all graph views. *Coordinated view maintenance* uses this view to update the reverse index,

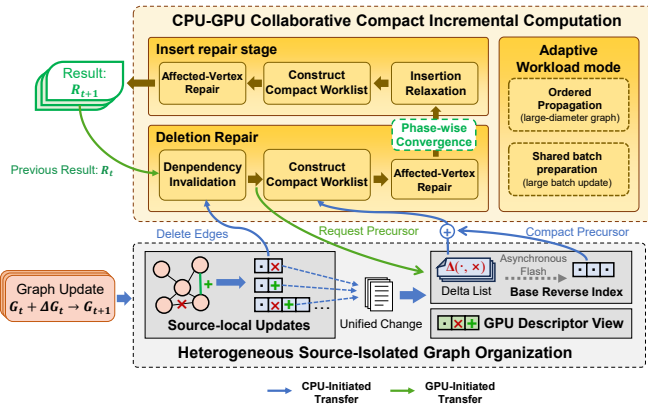


Fig. 2. The Overview of CoIncGraph System.

publish GPU descriptors, and repair cached adjacency only for changed sources.

1) *Source-Isolated Adjacency Blocks*: Relocation in shared arrays arises because the adjacency of different sources shares one address space, so the space required by one source is obtained by moving its neighbors. We therefore make the source the unit of allocation. The graph resides in pinned host memory mapped into the GPU address space, and each source stores its neighbors in an independently allocated, contiguous block with spare capacity, which the GPU locates through the source’s adjacency descriptor. An insertion appends neighbors in place when the block has sufficient capacity; otherwise, the CPU allocates a larger block, copies only this source’s adjacency, and appends the new neighbors. A deletion removes the requested edges that exist and compacts the remaining neighbors within the same block.

This organizational approach ensures that the adjacency relationships of a source change only when the source itself updates. The descriptors and cached adjacency of all other sources therefore remain valid, which makes it safe to publish GPU-side changes only for changed sources. As illustrated in Fig. 3, expanding w ’s block leaves all other blocks and descriptors untouched, whereas the same insertion in a PMA shifts and rebalances unrelated elements. Since each block keeps its neighbors contiguous, the coalesced zero-copy access of compact layouts is preserved.

2) *Unified Change View*: Source isolation bounds where adjacency moves, but each batch must still be reflected in several graph views: the host outgoing adjacency for traversal, a host reverse index that provides incoming adjacency for deletion repair, and the GPU descriptors and cached adjacency used by computation. Deriving the changes of each view independently from the update requests would repeat the same validation, and a deletion request that matches no existing edge could even make the views inconsistent. We instead resolve the requests once against the outgoing adjacency and record the outcome in a *unified change view*, which consists of effective edge-change records and changed-source lists generated in each update phase. The reverse index consumes the edge-

figures/view.pdf

Fig. 3. Source-Isolated Adjacency Blocks and Unified Change View.

change records, while GPU descriptor and cache maintenance consume the changed-source lists.

The CPU groups requests by source and determines their effects against the graph state at the start of each phase. The deletion operation removes existing edges by compacting the affected list. The insertion operation first determines whether the current block requires expansion before performing the addition. Grouping reduces repeated movement within lists, complementing source isolation across lists and allowing outgoing-adjacency updates for different sources to proceed in parallel. Preparation records the number of edges actually removed or added for each source–destination pair and identifies changed sources. Unsuccessful deletions therefore produce no change records.

For example, suppose u has neighbors $[a, a, b]$ and receives three deletion requests for (u, a) . Deletion removes the two existing copies, produces $[b]$, and emits $(u, a, -2)$ for reverse-index maintenance, while the unmatched third deletion request triggers no maintenance operation in any view. Then, the edges (w, c) and (w, d) are inserted, triggering an expansion and emitting $(w, c, +1)$ and $(w, b, +1)$. Consequently, vertex v remains unaffected by any operations involving other vertices throughout this entire phase, as illustrated in Fig. 3.

3) *Coordinated View Maintenance*: We coordinate reverse-index, GPU descriptor, and cache maintenance through the unified change view, avoiding repeated update validation and unnecessary data movement. The reverse index groups effective edge changes by destination and aggregates them by source–destination pair. Each destination maintains a sorted base list of incoming sources and delta records tracking net changes in edge multiplicity. Updates modify only the

deltas, removing zero-valued records rather than rewriting the base list. Queries recover current predecessors by adding each source's delta to its base multiplicity and retaining positive counts. For example, base list $[u, u, w]$ with deltas $[(u, -2), (v, +1)]$ yields incoming sources $[v, w]$. This representation supports deletion repair without rechecking input requests or reconstructing incoming adjacency from outgoing lists.

Source isolation further enables selective GPU descriptor and cache updates. Let S^- and S^+ denote sources changed during deletion and insertion. After insertion, the CPU publishes one descriptor record per source in $S = S^- \cup S^+$, containing its final adjacency location, degree, and version. Sources outside S retain valid descriptors, and unsuccessful deletions generate no publication records. With 24-byte records, descriptor traffic is $24|S|$ bytes per batch rather than proportional to the total vertex count. Cache repair likewise targets only changed sources: each cached list is refreshed in place if it fits, relocated to unused cache-tail space otherwise, or invalidated if space is insufficient. Each repair copies the source's entire current adjacency, while other cached lists remain in place. Incoming-neighbor transfers and host-adjacency accesses incur separate costs.

To preserve consistent access, descriptor and cache updates execute in one CUDA stream, subsequent computation waits for completion, and replaced host blocks are reclaimed only after earlier GPU readers finish. After computation, the hotness-based policy selects the next cache set. If it matches the previously published set and all selected entries remain valid, the system skips eviction, compaction, and loading; otherwise, it performs normal replacement. Coordinating selective publication with local cache repair thus avoids full descriptor reloads and unnecessary cache reorganization while preserving adaptation to changing workloads.

B. CPU-GPU Collaborative Compact Computation

As discussed in Section II-B, the GPU cannot locate the predecessors of invalidated vertices, so their recovery is merged into insertion processing, which reactivates all vertices and rebuilds worklists over the entire graph. The key idea of our incremental computation is to complete deletion repair on the graph after deletion before applying any insertion, using the incoming edges of affected vertices supplied by the CPU. Once deletion repair converges, every result is correct for the graph after deletion; any further change can then originate only from destinations improved by inserted edges, so insertion propagation needs to start only from these destinations. Without this decoupling, starting from inserted edges alone would miss alternative paths through inactive predecessors, such as (x, v) in Section II-B. Building on this property, we design two mechanisms. *Collaborative deletion repair* lets the GPU identify the affected vertices, while the CPU prepares their incoming edges from the host-resident data that is not available on the GPU, confining repair to the affected region. *Collaborative insertion propagation* seeds propagation

Algorithm 1 CPU–GPU cooperative processing of an update batch.

Input: Graph G_t , vertex results, deletions D_t , insertions I_t

Output: Graph G_{t+1}^{+E} and its converged vertex results

```

1: CPU:  $P \leftarrow \text{GROUPBYSOURCE}(D_t, I_t)$ 
   Step 1: Dependency invalidation (GPU)
2:  $A \leftarrow \text{INVALIDATEANDAPPEND}(G_t, D_t)$   $\triangleright$  Compact worklist
3:  $\text{RESETRESULTS}(A)$ ; transfer  $A$  to the CPU
   Step 2: Incoming-edge preparation (CPU)
4:  $S^- \leftarrow \text{APPLYDELETIONSANDUPDATEREVERSE}(P)$ 
5:  $I(A) \leftarrow \text{MATERIALIZINGINCOMING}(A)$ ; transfer  $I(A)$  to the GPU
   Step 3: Deletion repair (GPU)
6: repeat
7:   for all  $v \in A$  in parallel do
8:      $\text{PULLRELAX}(v, I(A))$   $\triangleright$  Eq. (2)
9:   until no result in  $A$  changes  $\triangleright$  Checked by the CPU
   Step 4: Topology publication (CPU–GPU)
10: CPU:  $S^+ \leftarrow \text{APPLYINSERTIONSANDUPDATEREVERSE}(P)$ 
11:  $\text{PUBLISHANDREPAIRCACHE}(S^- \cup S^+)$ ; wait for completion
   Step 5: Initial worklist construction (GPU)
12:  $L \leftarrow \text{RELAXINSERTIONSANDAPPEND}(I_t)$ 
   Step 6: Insertion propagation (one cooperative GPU kernel)
13: while  $L \neq \emptyset$  do
14:    $L_{\text{next}} \leftarrow \emptyset$ 
15:   for all  $u \in L$  in parallel do
16:     if  $\text{COMMITLATESTIMPROVEMENT}(u)$  then
17:       for all outgoing edges  $(u, v)$  in  $G_{t+1}^{+E}$  do
18:         if  $\text{ATOMICRELAX}(u, v)$  succeeds then
19:            $\text{APPENDONCE}(v, L_{\text{next}})$ 
20:    $\text{GRIDSYNCHRONIZE}$ ;  $\text{SWAP}(L, L_{\text{next}})$ 

```

only with improved destinations and builds compact worklists directly from successful updates.

1) *Collaborative Deletion Repair*: We describe the process using SSSP with positive edge weights (BFS uses unit weights). A vertex result is its current distance estimate, denoted by $d(v)$, and a better result corresponds to a smaller estimate in this example. For a batch of effective deletions D_t and insertions I_t , let $G_t = (V, E_t)$ be the previous graph. The graph after deletion is $G_{t+1}^{-E} = (V, E_t \setminus D_t)$, and the final graph is $G_{t+1}^{+E} = (V, (E_t \setminus D_t) \cup I_t)$. Algorithm 1 summarizes the CPU–GPU cooperative workflow of a batch, which starts by grouping the requests by source (Line 1).

1 Dependency invalidation (Lines 2–3). Following dependency-memoization, a deletion invalidates the results of all vertices whose dependencies pass through the deleted edge, and these vertices must be reset before repair. Collecting them by scanning vertex marks across all graph partitions would examine the entire graph in every round, although the invalidated region is typically small. We instead build a compact worklist directly as vertices are marked.

The GPU initializes an empty worklist L_{del} and examines the deleted edges in parallel. For each deleted edge (u, v) , it marks v if the current result of v depends on that edge, and only the thread that first marks v atomically reserves a slot and appends v to L_{del} . Each subsequent round scans, in G_t , the outgoing edges of the vertices appended in the preceding round and appends newly marked dependents in the same

way, yielding a breadth-first traversal of dependencies. When a round appends no vertex, L_{del} holds exactly the affected vertices, denoted by A : each appears once in consecutive slots, and unaffected vertices occupy no slot. The GPU resets the stored and tentative results of A to ∞ and transfers their IDs to the CPU. A thus serves as the interface between the two processors: it determines exactly which incoming edges the CPU prepares for repair.

2 Preparing incoming edges (Lines 4–5). An affected vertex may regain its result through an unaffected predecessor whose result has not changed. For example, after deleting (u, v) , the remaining edge (x, v) may restore v even though x is inactive. Repair therefore requires the remaining incoming edges of the vertices in A :

$$I(A) = \{(u, v) \in E(G_{t+1}^{-E}) \mid v \in A\}. \quad (1)$$

The GPU supplies only the IDs in A , and the CPU obtains their predecessors from the reverse index (Section III-A3), which already reflects the effective deletions of the batch. For each $v \in A$, the CPU merges the base list and the delta records of v in source order and retains sources with positive multiplicity. Predecessor discovery is thus confined to the incoming records of A , without scanning the outgoing adjacency of any source.

For a large A , CPU workers partition the destinations in A into disjoint groups. Each worker merges the incoming lists of its group into a private buffer and records their lengths; a prefix sum over the lengths then assigns disjoint ranges in the output array, and workers copy their buffers in parallel without synchronizing on a shared append buffer. Each destination's records are therefore merged only once.

The result stores the incoming list of each destination contiguously, with offsets delimiting the lists in the order of A . The CPU transfers these offsets and source IDs to the GPU, while vertex results remain on the GPU. With M_A incoming entries, this representation occupies $O(|A| + M_A)$ space, and its construction work is proportional to $|A|$ plus the base and delta records examined for these destinations. *Consequently, the data prepared by the CPU and transferred for repair is determined by the affected region rather than by the graph size.*

3 Repairing vertex results (Lines 6–9). Using the prepared incoming edges, the GPU repeatedly improves the result of each $v \in A$ with candidates from its predecessors. For SSSP, this pull-based update is:

$$d(v) \leftarrow \min \left\{ d(v), \min_{(u,v) \in I(A)} [d(u) + w(u, v)] \right\}. \quad (2)$$

A warp cooperatively scans the incoming list of each affected vertex, and improved results of affected vertices serve as candidates for their affected successors in later rounds. Vertices without any remaining path keep the result ∞ .

After step 3, all results are correct for G_{t+1}^{-E} , while repair has examined only A and $I(A)$. This stable state is the precondition of the compact insertion propagation in Section III-B2:

since no invalidated result remains, the inserted edges are the only source of further changes.

2) Collaborative Insertion Propagation: 4 Publishing the updated graph (Lines 10–11). After deletion repair, the CPU applies the insertions, updates the reverse index, and forms the changed-source set $S = S^- \cup S^+$. It then publishes the final descriptors of S and repairs their cached adjacency (Section III-A3). Once publication completes, the GPU traverses the final graph without a full descriptor reload.

5 Building the initial worklist (Line 12). Since deletion repair has already restored all results, only destinations improved by inserted edges can initiate further changes. After publication, GPU threads relax the inserted edges in parallel with atomic minimum updates, and each destination whose tentative result improves is appended at most once to the initial worklist, using the same append-on-change construction as step 1. The initial worklist therefore contains at most $|I_t|$ vertices instead of all $|V|$ vertices, and the subsequent adjacency traversal starts only from vertices whose results have actually improved.

6 Propagating result improvements (Lines 13–20). For each vertex in the current worklist, a GPU thread block checks whether its latest tentative result is better than its stored result. If so, it commits the improvement (Line 16) and reads the vertex's contiguous outgoing adjacency through its published descriptor, either from the hot-subgraph cache or from host memory via zero-copy access. Whenever an edge improves its destination's tentative result, the destination is appended to the next worklist (Lines 18–19). An atomic minimum preserves the best concurrent update, and a per-vertex tag containing the batch and round admits each destination at most once per round, while a later improvement may admit it again in a subsequent round. Successful updates thus construct the next worklist directly, and no round scans unrelated vertices to rebuild worklists.

With compact worklists, later rounds often contain only a few active vertices, so per-round host coordination, including kernel launches, worklist exchange, and convergence checks, can outweigh the useful work. We therefore execute insertion propagation within a single cooperative GPU kernel. The GPU maintains the current and next worklists, uses grid-wide barriers to complete pending writes before exchanging them (Line 20), and terminates when the next worklist is empty; the CPU only waits for the kernel to finish. This removes host orchestration and repeated kernel launches between insertion rounds.

Let X_k be the vertices whose results improve and whose outgoing edges are examined in round k . The edge work is

$$W_{\text{expand}} = \sum_k \sum_{u \in X_k} \deg_{G_{t+1}^{+E}}^+(u). \quad (3)$$

Vertices without improvements never read their adjacency, which reduces both edge processing and requests to host memory. Repeated improvements of the same vertex may still cause repeated reads, which ordered propagation reduces (Section III-B3).

TABLE II
DATASET USED FOR THE EXPERIMENT.

Graph	Vertices	Edges	Degree	Diameter
Orkut (OK) [13]	3,072,441	117,185,083	76.28	9
Wikipedia (WK) [14]	13,593,032	437,217,424	32.16	10
Twitter-2009 (TW) [15]	52,579,682	1,963,263,821	37.34	18
Friendster (FS) [13]	65,608,366	1,806,067,135	55.06	32
Europe OSM (EU) [16]	50,912,018	54,054,660	2.12	30,102
USA Road (USA) [17]	23,947,347	28,854,312	2.41	8,440

3) *Ordered Propagation*: Compact worklists exclude vertices whose results do not change, but they cannot prevent the same vertex from being activated repeatedly by successively better results. In large-diameter graphs such as road networks, updates propagate over many rounds, and propagating every intermediate improvement immediately triggers repeated scans of downstream adjacency that are later superseded. For instance, in algorithms where a smaller value is preferable, ordered propagation prioritizes vertices with smaller tentative values, so that deferred vertices can receive further improvements before scanning their outgoing edges. The GPU assigns each pending vertex with tentative value r to bucket $\lfloor r/\Delta \rfloor$, where $\Delta > 0$ is the bucket width, processes the smallest nonempty bucket, and retains the other vertices for later rounds. This ordering changes only the order in which pending vertices are served in deletion repair (Lines 6–9) and insertion propagation (Lines 13–20). We enable this ordering for large-diameter graphs. It reduces redundant edge scans and host-memory accesses at the cost of more rounds, and is therefore not used for low-diameter graphs, where propagation converges in a few rounds.

IV. EVALUATION

A. Experiment Setup

1) *Experimental Environment*: We implement our system in C++ and CUDA and evaluate it on a server with two Intel Xeon Gold 5218R processors (80 physical cores in total) and an NVIDIA V100 GPU. Each CPU has a base frequency of 2.10 GHz. The V100 has 80 streaming multiprocessors (5,120 CUDA cores) and 16 GB of global memory. The GPU connects to the host via PCIe 3.0 $\times 16$, with a theoretical peak bandwidth of approximately 16 GB/s per direction. We compile the code with NVIDIA NVCC 13.0 and GCC 13.3.0 using the `-O3` optimization flag.

2) *Datasets*: We evaluate CoIncGraph on six real-world graphs whose statistics are summarized in Table II. OK [13], WK [14], TW [15], and FS [13] are social networks with power-law degree distributions, whereas EU and USA are road networks with large diameters. Following Grapin and GraphBolt [4], [1], we uniformly sample edges without replacement and assign half to insertions and half to deletions. We remove the insertion edges from the original graph to form the initial graph G_0 , then randomly interleave the updates and partition them into ten equal-sized batches, each containing equal numbers of insertions and deletions.

3) *Baselines*: We compare CoIncGraph with three state-of-the-art systems: Ingress, SHGraph, and Grapin. SHGraph [3], Gunrock’s dynamic graph module, uses per-vertex Slab Hash tables to support parallel graph updates and queries in GPU memory. Ingress [2] performs CPU-based incremental graph processing and automatically selects a memoization policy (memoization-free, memoization-path, memoization-vertex, or memoization-edge) according to the algorithm properties, so that only the memoized states required by the algorithm are maintained and revised. Grapin [4] supports CPU–GPU co-operative out-of-memory (OOM) streaming graph processing by maintaining graph topology in host memory and executing incremental computation on the GPU. It combines GPU hot-subgraph caching with optimized zero-copy access to reduce CPU–GPU data movement.

4) *Methodology*: We evaluate all systems on [graph algorithms] using identical initial graphs and update sequences. We measure end-to-end processing time and update throughput (processed update edges per second), including graph updates, incremental computation, and CPU–GPU transfers where applicable, but excluding initial graph loading and preprocessing. Speedup is the ratio of baseline processing time to that of CoIncGraph. We report the arithmetic mean of five independent runs, each starting from the same initial graph. We also examine sensitivity to [batch size and other parameters] and quantify individual optimizations through ablation studies.

B. Overall Performance

We use ten consecutive batches of 1K, 10K, and 100K edge updates to evaluate the efficiency of CoIncGraph, as shown in Table III. CoIncGraph achieves the shortest execution time in 47 of the 48 cases; the only exception is SSSP on OK with 100K updates per batch, where the in-memory SHGraph is faster.

Comparison with CPU-based systems. Benefiting from GPU parallelism and compact incremental computation, CoIncGraph outperforms Ingress in all 48 cases, achieving speedups ranging from $4.2\times$ to $1979.0\times$ (geometric mean $127.8\times$). Note that the reported time of Ingress excludes its topology maintenance, so these speedups are conservative. Although Ingress also confines computation to the affected states through memoization, it revises the memoized states with limited CPU parallelism and memory bandwidth, which cannot match the massive parallelism of the GPU. The speedup is most significant for BFS and SSSP (geometric mean $387.4\times$ and $384.9\times$), followed by CC ($124.5\times$). For these algorithms, Ingress maintains path or vertex dependencies on the CPU, and the revision of invalidated results requires multiple rounds of traversal over the host-resident graph. PR achieves a lower speedup (geometric mean $14.4\times$), as Ingress executes it with memoization-free, delta-based accumulation, which propagates only the changes caused by updates without maintaining any dependency. Moreover, the advantage of CoIncGraph grows with graph size. On the two small graphs, CoIncGraph achieves a geometric-mean speedup of $104.5\times$, while on the

TABLE III
EXECUTION TIME (IN SECONDS) ACROSS TEN BATCHES OF 1K, 10K, AND 100K EDGE UPDATES. OOM INDICATES RUNNING OUT OF GPU MEMORY;
THE BEST RESULT IN EACH COLUMN IS IN BOLD.

Alg.	System	Batch size = 1K				Batch size = 10K				Batch size = 100K			
		OK	WK	TW	FS	OK	WK	TW	FS	OK	WK	TW	FS
BFS	Ingress	9.60	170	750	1350	11.1	183	820	1361	12.7	186	866	1360
	SHGraph	0.62	OOM	OOM	OOM	0.58	OOM	OOM	OOM	0.67	OOM	OOM	OOM
	Grabin	1.07	2.78	13.2	14.4	1.37	3.21	13.8	14.5	2.44	4.58	16.2	16.0
	CoIncGraph	0.034	0.10	1.05	1.13	0.079	0.15	1.73	1.33	0.63	1.08	4.25	2.78
SSSP	Ingress	10.2	194	764	1382	11.7	208	803	1402	13.5	208	989	1445
	SHGraph	0.34	OOM	OOM	OOM	0.32	OOM	OOM	OOM	0.24	OOM	OOM	OOM
	Grabin	1.09	2.83	11.8	13.4	1.64	3.54	12.5	13.9	2.80	5.24	15.2	15.5
	CoIncGraph	0.036	0.098	1.25	1.26	0.097	0.17	2.00	1.46	0.57	0.96	4.66	3.15
CC	Ingress	12.8	113	591	1067	12.8	119	627	1105	13.1	140	754	1122
	SHGraph	0.78	OOM	OOM	OOM	0.77	OOM	OOM	OOM	0.71	OOM	OOM	OOM
	Grabin	1.24	2.77	12.3	12.8	1.53	3.22	12.6	13.1	1.95	5.08	14.5	14.0
	CoIncGraph	0.036	0.78	3.16	3.55	0.079	1.31	4.21	4.65	0.53	3.38	8.57	10.8
PR	Ingress	7.17	64.8	310	107	9.01	72.2	319	137	11.5	97.5	374	180
	SHGraph	4.51	OOM	OOM	OOM	4.49	OOM	OOM	OOM	4.51	OOM	OOM	OOM
	Grabin	4.64	21.2	78.9	27.9	5.24	22.6	83.7	32.0	6.41	25.5	99.6	40.5
	CoIncGraph	0.14	2.21	15.9	3.17	0.39	6.36	35.3	8.68	1.12	12.2	90.0	30.0

two billion-scale graphs, it achieves $156.2\times$, where the larger affected regions expose more parallelism to the GPU.

Comparison with GPU in-memory systems. SHGraph keeps the entire graph and its per-vertex slab hash tables in device memory, and thus runs out of memory on WK, TW, and FS. In contrast, CoIncGraph maintains graph topology in host memory and succeeds in all cases. On OK, which fits in GPU memory, CoIncGraph outperforms SHGraph in 11 of the 12 cases with speedups of up to $32.86\times$ (geometric mean $5.53\times$). The advantage is most pronounced for small batches: with 1K updates per batch, CoIncGraph is $9.44\times$ to $32.86\times$ faster than SHGraph. The execution time of SHGraph remains nearly constant across batch sizes (e.g., 4.49–4.51 s for PR), indicating that its cost is dominated by graph-wide processing rather than by the updates themselves, whereas CoIncGraph confines computation to the affected vertices. As the batch size increases to 100K, the affected region expands, and CoIncGraph has to fetch more uncached adjacency from host memory over PCIe, whose bandwidth is more than an order of magnitude lower than that of the GPU’s on-board memory. Consequently, the speedup narrows to $1.06\times$ – $4.02\times$ for BFS, CC, and PR, and SHGraph is $2.38\times$ faster than CoIncGraph for SSSP. This is expected, as SHGraph represents an in-memory upper bound that avoids CPU–GPU data movement entirely but cannot scale beyond GPU memory capacity.

Comparison with CPU–GPU out-of-memory systems. CoIncGraph outperforms Grabin in all 48 cases, achieving speedups ranging from $1.11\times$ to $34.30\times$ (geometric mean $6.50\times$). Grabin merges the recovery of invalidated vertices into insertion processing, which reactivates all vertices and rebuilds worklists over the entire graph, and its shared-array topology maintenance may relocate unrelated adjacency and refresh their GPU descriptors. As a result, the execution time of Grabin is largely insensitive to the number of updates: increasing the batch size from 1K to 100K (i.e., $100\times$ more updates) raises its execution time by only $1.10\times$ – $2.57\times$. In contrast, CoIncGraph confines topology publication to

changed sources (Section III-A) and restricts deletion repair and insertion propagation to affected vertices (Section III-B), so its cost follows the affected region rather than the graph size.

Impact of batch size. The speedup over Grabin decreases with the batch size, with geometric means of $12.35\times$, $7.54\times$, and $2.96\times$ for 1K, 10K, and 100K updates, respectively. Small batches affect only a small portion of the graph, so the graph-wide work of Grabin dominates its execution time, while CoIncGraph processes only a few affected vertices. Larger batches enlarge the affected region, increasing the incoming edges that the CPU prepares and transfers for deletion repair and the outgoing adjacency traversed during insertion propagation. Nevertheless, CoIncGraph remains faster than Grabin in all 16 cases with 100K updates ($1.11\times$ – $5.76\times$).

Impact of algorithms. BFS and SSSP achieve the highest speedups (geometric mean $10.39\times$ and $9.80\times$), compared with $3.77\times$ for CC and $4.66\times$ for PR. For BFS and SSSP, a deleted edge invalidates only the vertices whose shortest paths pass through it, and inserted edges improve only a few destinations, so phase decoupling keeps both deletion repair and insertion propagation compact. In CC, a label is shared by all vertices in a component, so deleting an edge on the dependency path of a label may invalidate a large portion of the component, enlarging the affected set and its incoming edges. PR is an accumulative algorithm in which each update perturbs the ranks of all reachable downstream vertices, and propagation continues until the residuals fall below the convergence threshold. The affected region therefore spreads over a large fraction of the graph, especially for large batches on TW, where the speedup drops to $1.11\times$.

Impact of graph size. The speedup is higher on the two smaller graphs (geometric mean $13.37\times$ on OK and $6.65\times$ on WK) than on the two billion-scale graphs ($4.04\times$ on TW and $4.98\times$ on FS). On larger graphs, the GPU hot-subgraph cache covers a smaller fraction of the adjacency, so a larger share of edge accesses is served from host memory through

zero-copy access. This PCIe-bound access cost is incurred by both systems and thus dilutes the reduction in redundant computation achieved by CoIncGraph.

C. Optimization Analysis

D. Scalability and Sensitivity Analysis

V. RELATED WORK

A. Dynamic Graph Processing on CPUs

CPU-based dynamic graph processing has primarily focused on efficient graph updates and incremental analytics. For graph storage, Sortledton, Spruce, DCSR, Bubble, and LSMGraph develop representations that support updates while retaining efficient access for analytics [18], [7], [19], [20], [8]. RisGraph combines graph updates with low-latency incremental analysis [21]. On the computation side, a small update often affects only a subset of the existing results. KickStarter, GraphBolt, and DZiG exploit this property to avoid recomputation over unaffected parts of the graph [10], [1], [22]. Tripoline, CommonGraph, and IncBoost further extend incremental processing to different query, snapshot, and update settings [23], [24], [25]. These techniques reduce redundant CPU computation, motivating efforts to exploit GPU parallelism for evolving graphs.

B. Dynamic Graph Processing on GPUs

GPU graph processing initially focused on static graphs. Gunrock, for example, provides parallel primitives for graph analytics [26]. For static graphs that exceed GPU memory, Subway transfers active subgraphs, EMOGI accesses host-resident edges, and HyTGraph adapts the data access strategy to the workload [11], [12], [27]. CGgraph retains a reusable subgraph on the GPU and distributes work between the CPU and GPU [28]. Processing evolving graphs adds further requirements: the graph representation must support updates without impeding traversal, while incremental execution must handle irregular and varying amounts of affected work.

GPU-resident dynamic graph systems primarily address the update and traversal tradeoff. GPMA supports parallel batches of edge updates using packed memory arrays [9]. Horner and faimGraph develop dynamic GPU graph structures [29], [30], while GASgraph uses a SubHPMA-based representation for streaming graph processing [31]. Their reliance on GPU-resident graph data limits the graph sizes they can accommodate. For graphs exceeding GPU memory, Grapin combines GPU incremental computation with hot-subgraph caching to reduce accesses to host-resident data [4]. CGCGraph co-executes concurrent snapshot analyses by processing shared graph data on the GPU and unshared data on the CPU [32]. POEGA uses proxy graphs to guide out-of-memory refinement and processes multiple snapshots concurrently [33]. CoIncGraph instead partitions computation affected by streaming updates between the CPU and GPU, coupling incremental execution with cooperative processing of graphs that exceed GPU memory.

VI. CONCLUSION

We presented CoIncGraph, a CPU–GPU cooperative system for incremental processing of out-of-memory streaming graphs. Its heterogeneous source-isolated graph organization preserves topological locality, limiting adjacency relocation and GPU index maintenance to changed sources. Collaborative deletion repair and insertion propagation restrict computation to affected vertices while coordinating CPU preparation with GPU execution. Ordered propagation and shared batch preparation further reduce redundant edge scans and CPU maintenance overhead. Experiments on six real-world graphs demonstrate that CoIncGraph outperforms Ingress, SHGraph, and Grapin. These results highlight the benefits of jointly exploiting update locality and CPU–GPU cooperation for efficient streaming graph processing.

REFERENCES

- [1] M. Mariappan and K. Vora, “Graphbolt: Dependency-driven synchronous processing of streaming graphs,” in *Proceedings of the Fourteenth EuroSys Conference 2019*, ser. EuroSys ’19. Association for Computing Machinery, 2019.
- [2] S. Gong, C. Tian, Q. Yin, W. Yu, Y. Zhang, L. Geng, S. Yu, G. Yu, and J. Zhou, “Automating incremental graph processing with flexible memoization,” *Proceedings of the VLDB Endowment*, vol. 14, no. 9, pp. 1613–1625, 2021.
- [3] M. A. Awad, S. Ashkiani, S. D. Porumbescu, and J. D. Owens, “Dynamic graphs on the gpu,” in *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, May 2020, pp. 739–748.
- [4] Q. Wang, Y. Yan, H. Tan, C. Chen, C. Zhao, J. Tian, J. Jiang, X. Cong, Y. Zhang, G. Yu, W.-F. Wong, and B. He, “Efficient graph data access for out-of-memory gpu streaming graph processing,” *Proc. VLDB Endow.*, vol. 18, no. 11, p. 3854–3867, Jul. 2025.
- [5] J. Shun and G. E. Blelloch, “Ligra: a lightweight graph processing framework for shared memory,” in *Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. Association for Computing Machinery, 2013, p. 135–146.
- [6] P. Pandey, B. Wheatman, H. Xu, and A. Buluc, “Terrace: A hierarchical graph container for skewed dynamic graphs,” in *Proceedings of the 2021 International Conference on Management of Data*, ser. SIGMOD ’21. Association for Computing Machinery, 2021, p. 1372–1385.
- [7] J. Shi, B. Wang, and Y. Xu, “Spruce: a fast yet space-saving structure for dynamic graph storage,” *Proc. ACM Manag. Data*, vol. 2, no. 1, Mar. 2024.
- [8] S. Yu, S. Gong, Q. Tao, S. Shen, Y. Zhang, W. Yu, P. Liu, Z. Zhang, H. Li, X. Luo, G. Yu, and J. Zhou, “Lsmgraph: A high-performance dynamic graph storage system with multi-level csr,” *Proc. ACM Manag. Data*, vol. 2, no. 6, Dec. 2024.
- [9] M. Sha, Y. Li, B. He, and K.-L. Tan, “Accelerating dynamic graph analytics on gpus,” *Proc. VLDB Endow.*, vol. 11, no. 1, p. 107–120, Sep. 2017.
- [10] K. Vora, R. Gupta, and G. Xu, “Kickstarter: Fast and accurate computations on streaming graphs via trimmed approximations,” in *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS ’17. Association for Computing Machinery, 2017, p. 237–251.
- [11] A. H. N. Sabet, Z. Zhao, and R. Gupta, “Subway: minimizing data transfer during out-of-gpu-memory graph processing,” in *Proceedings of the Fifteenth European Conference on Computer Systems*, ser. EuroSys ’20. Association for Computing Machinery, 2020.
- [12] S. W. Min, V. S. Mailthody, Z. Qureshi, J. Xiong, E. Ebrahimi, and W.-m. Hwu, “Emogi: efficient memory-access for out-of-memory graph-traversal in gpus,” *Proc. VLDB Endow.*, vol. 14, no. 2, p. 114–127, Oct. 2020.
- [13] J. Yang and J. Leskovec, “Defining and evaluating network communities based on ground-truth,” in *Proceedings of the ACM SIGKDD Workshop on Mining Data Semantics*, ser. MDS ’12. New York, NY, USA: Association for Computing Machinery, 2012. [Online]. Available: <https://doi.org/10.1145/2350190.2350193>

- [14] J. Kunegis, “Konect: the koblenz network collection,” in *Proceedings of the 22nd International Conference on World Wide Web*, ser. WWW ’13 Companion. New York, NY, USA: Association for Computing Machinery, 2013, p. 1343–1350. [Online]. Available: <https://doi.org/10.1145/2487788.2488173>
- [15] M. Cha, H. Haddadi, F. Benevenuto, and K. P. Gummadi, “Measuring user influence in twitter: The million follower fallacy,” vol. 14, 06 2010.
- [16] D. A. Bader, H. Meyerhenke, P. Sanders, and D. Wagner, *Graph Partitioning and Graph Clustering: 10th DIMACS Implementation Challenge Workshop*. American Mathematical Society, 2013, vol. 588.
- [17] C. Demetrescu, A. V. Goldberg, and D. Johnson, *Shortest Paths: 9th DIMACS Implementation Challenge*. American Mathematical Society, 2009, vol. 74.
- [18] P. Fuchs, D. Margan, and J. Giceva, “Sortledton: a universal, transactional graph data structure,” *Proc. VLDB Endow.*, vol. 15, no. 6, p. 1173–1186, Feb. 2022.
- [19] Y. Feng, Y. Zhang, S. Li, P. Zhang, and H. Cao, “Dcsr: A dual-layer graph storage structure for hybrid transactional and analytical processing,” in *2025 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA)*, 2025, pp. 625–632.
- [20] L. Deng, Y. Li, Z. Zhang, Y. Xu, and J. C. S. Lui, “Bubble: Towards scalable evolving graph processing via mini-batch sorting,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC ’25. Association for Computing Machinery, 2025, p. 1553–1571.
- [21] G. Feng, Z. Ma, D. Li, S. Chen, X. Zhu, W. Han, and W. Chen, “Risgraph: A real-time streaming system for evolving graphs to support sub-millisecond per-update analysis at millions ops/s,” in *Proceedings of the 2021 International Conference on Management of Data*, ser. SIGMOD ’21. Association for Computing Machinery, 2021, p. 513–527.
- [22] M. Mariappan, J. Che, and K. Vora, “Dzig: sparsity-aware incremental processing of streaming graphs,” in *Proceedings of the Sixteenth European Conference on Computer Systems*, ser. EuroSys ’21. Association for Computing Machinery, 2021, p. 83–98.
- [23] X. Jiang, C. Xu, X. Yin, Z. Zhao, and R. Gupta, “Tripoline: generalized incremental graph processing via graph triangle inequality,” in *Proceedings of the Sixteenth European Conference on Computer Systems*, ser. EuroSys ’21. Association for Computing Machinery, 2021, p. 17–32.
- [24] M. Afarin, C. Gao, S. Rahman, N. Abu-Ghazaleh, and R. Gupta, “Commongraph: Graph analytics on evolving data,” in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. Association for Computing Machinery, 2023, p. 133–145.
- [25] X. Yin, Z. Zhao, and R. Gupta, “Incboost: Scaling incremental graph processing for edge deletions and weight updates,” in *Proceedings of the 2024 ACM Symposium on Cloud Computing*, ser. SoCC ’24. Association for Computing Machinery, 2024, p. 915–932.
- [26] Y. Wang, A. Davidson, Y. Pan, Y. Wu, A. Riffel, and J. D. Owens, “Gunrock: a high-performance graph processing library on the gpu,” in *Proceedings of the 21st ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, ser. PPoPP ’16. Association for Computing Machinery, 2016.
- [27] Q. Wang, X. Ai, Y. Zhang, J. Chen, and G. Yu, “Hytgraph: Gpu-accelerated graph processing with hybrid transfer management,” in *2023 IEEE 39th International Conference on Data Engineering (ICDE)*, 2023, pp. 558–571.
- [28] P. Cui, H. Liu, B. Tang, and Y. Yuan, “Cggraph: An ultra-fast graph processing system on modern commodity cpu-gpu co-processor,” *Proc. VLDB Endow.*, vol. 17, no. 6, p. 1405–1417, Feb. 2024.
- [29] F. Busato, O. Green, N. Bombieri, and D. A. Bader, “Hornet: An efficient data structure for dynamic sparse graphs and matrices on gpus,” in *2018 IEEE High Performance extreme Computing Conference (HPEC)*, 2018, pp. 1–7.
- [30] M. Winter, D. Mlakar, R. Zayer, H.-P. Seidel, and M. Steinberger, “faimgraph: high performance management of fully-dynamic graphs under tight memory constraints on the gpu,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*, ser. SC ’18. IEEE Press, 2019.
- [31] C. Wang, Y. Zhang, H. Cao, X. An, and X. Ye, “GASgraph: A gpu-accelerated streaming graph processing system based on SubHPMAs,” in *Advanced Parallel Processing Technologies: 16th International Symposium, APPT 2025, Athens, Greece, July 13-16, 2025, Proceedings*. Springer-Verlag, 2025, p. 284–296.
- [32] Y. Sun, J. Zhang, H. Cao, Y. Zhang, X. An, J. Huang, and X. Ye, “Cgc-graph: Efficient cpu-gpu co-execution for concurrent dynamic graph processing,” *ACM Trans. Archit. Code Optim.*, vol. 22, no. 3, Sep. 2025.
- [33] Y. Zhang, J. Huang, X. Yin, J. Qiu, H. Xu, and C. J. Xue, “Efficient GPU-Centric evolving graph processing at scale,” in *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*. USENIX Association, Jul. 2026, pp. 1585–1604.